Parallel and Distributed Computing

Lecture 8

Bazam-e-Jahan
Lecturer
The University of Faisalabad

Failure Models

- Failure models describe the types of failures that can occur in a distributed system. Understanding these models is crucial for designing fault-tolerant systems.

Cont…

Crash Failures

- **Description**: A node stops functioning and does not respond to messages.
- **Example**: A server crashes due to a hardware failure.
- **Impact**: The node is unavailable until it recovers.

Omission Failures

- **Description**: A node fails to send or receive messages.
- **Types**:
  - **Send Omission**: A node fails to send a message.
  - **Receive Omission**: A node fails to receive a message.
- **Example**: Network congestion causes messages to be dropped.
- **Impact**: Communication between nodes is disrupted.

Cont…

Byzantine Failures

- **Description**: A node behaves arbitrarily, sending incorrect or malicious messages.
- **Example**: A compromised node sends false data to other nodes.
- **Impact**: The system may produce incorrect results.

Timing Failures

- **Description**: A node fails to meet timing constraints (e.g., responding too late).
- **Example**: A node in a real-time system misses a deadline.
- **Impact**: The system may violate timing guarantees

Design Issues for Fault Tolerance

Designing fault-tolerant distributed systems involves addressing several key issues:

1. **Failure Detection**:
    1. Identifying when a node or component has failed.
    2. Techniques include **heartbeat messages** and **timeouts**.
2. **Replication**:
    1. Maintaining multiple copies of data or services to ensure availability.
    2. Challenges include **consistency** and **coordination**.
3. **Consensus**:
    1. Ensuring that all nodes agree on a value or decision.

Design Issues for Fault Tolerance

1. **Recovery**:
    1. Restoring the system to a consistent state after a failure.
    2. Techniques include **checkpointing** and **log-based recovery**.
2. **Redundancy**:
    1. Adding extra components to handle failures.
    2. Examples include **hot standby** and **failover mechanisms**.